Actividad Sesión No. 2

¿Cuál algoritmo escala mejor?

Análisis de algoritmos y notación Big O

Dato	Información
Estudiante	Juan David López Ebratt
Asignatura	Estructura de Datos I
Programa	Ingeniería de Software
Docente	Mg. Sandra Esperanza Sánchez Barragán
Lenguaje / Entorno	Java / Visual Studio Code

1. Clasificación de fragmentos (Sección 6)

Fragmento	Operaciones aproximadas	Big O	Justificación
Acceder a vector[5]	1	O(1)	Se accede directamente a una posición por su índice; el costo no depende del tamaño del arreglo.
Recorrer un arreglo	n	O(n)	Un solo ciclo visita cada elemento exactamente una vez; el trabajo crece proporcional a n.
Dos ciclos anidados	$n \times n = n^2$	$O(n^2)$	Por cada iteración del ciclo externo (n veces), el interno se ejecuta n veces.
Dividir n entre 2 repetidamente	$\log_2 n$	O(log n)	En cada iteración el problema se reduce a la mitad, por eso hay pocas iteraciones (ej.: n = 1.000 → unas 10).
Ciclo n + reducción logarítmica	$n \times \log_2 n$	O(n log n)	Un ciclo externo de n iteraciones que contiene una reducción a la mitad (≈ log n pasos) por cada iteración (Ejercicio 5).

2. Explicación línea por línea de los ejercicios 1 a 5

Ejercicio 1 – O(1)

Línea de código	Qué hace
`public static void mostrarPrimero(int[] datos) {`	Declara un método estático (se llama sin crear objetos) que no devuelve nada (void) y recibe un arreglo de enteros llamado datos.
`  System.out.println(datos[0]);`	Imprime en consola el elemento de la posición 0. Es una sola consulta por índice.
`}`	Cierra el método.

Complejidad: O(1). Sin importar si el arreglo tiene 10 o 1.000.000 de elementos, solo se hace un acceso.

Ejercicio 2 – O(n)

Línea de código	Qué hace
`public static void mostrarElementos(int[] datos) {`	Declara el método que recibe el arreglo datos.
`  for (int i = 0; i < datos.length; i++) {`	Ciclo for: i empieza en 0, continúa mientras i sea menor que la longitud del arreglo (datos.length) y aumenta en 1 en cada vuelta. Se ejecuta n veces.
`System.out.println(datos[i]);`	En cada vuelta imprime el elemento de la posición i.
`  }`	Cierra el ciclo.
`}`	Cierra el método.

Complejidad: O(n). El ciclo recorre cada elemento una vez; con n = 1.000 hace unas 1.000 iteraciones.

Ejercicio 3 – O(n²)

Línea de código	Qué hace
`public static void compararTodos(int[] datos) {`	Declara el método que recibe el arreglo datos.
`  for (int i = 0; i < datos.length; i++) {`	Ciclo externo: recorre las posiciones i de 0 a n − 1 (n vueltas).
`    for (int j = 0; j < datos.length; j++) {`	Ciclo interno: por CADA valor de i, j recorre de nuevo de 0 a n − 1 (n vueltas).
`System.out.println(datos[i] + " - " + datos[j]);`	Imprime el par de elementos datos[i] y datos[j] separados por un guion. Esta línea se ejecuta n × n veces.
`    }`	Cierra el ciclo interno.
`  }`	Cierra el ciclo externo.
`}`	Cierra el método.

Complejidad: O(n²). Con n = 1.000 se ejecutan 1.000 × 1.000 = 1.000.000 de impresiones.

Ejercicio 4 – O(log n)

Línea de código	Qué hace
`public static void reducir(int n) {`	Declara un método que recibe un número entero n.
`  while (n > 1) {`	Ciclo while: se repite mientras n sea mayor que 1.
`    n = n / 2;`	Divide n entre 2 (división entera). Así el problema se reduce a la mitad en cada vuelta.
`    System.out.println(n);`	Imprime el nuevo valor de n.
`  }`	Cierra el ciclo.
`}`	Cierra el método.

Complejidad: O(log n). Por ejemplo, con n = 1.000 los valores son 500, 250, 125, 62, 31, 15, 7, 3, 1: unas 10 iteraciones ($\log_2 1.000 \approx 10$).

Ejercicio 5 – O(n log n)

Línea de código	Qué hace
`public static void ejemploNLogN(int n) {`	Declara el método que recibe el entero n.
`  for (int i = 0; i < n; i++) {`	Ciclo externo: se ejecuta n veces.
`    int valor = n;`	En cada vuelta externa se reinicia la variable valor con el valor de n.
`    while (valor > 1) {`	Ciclo interno: se repite mientras valor sea mayor que 1.
`      valor = valor / 2;`	Divide valor entre 2; por eso el ciclo interno dura aproximadamente $\log_2$ n vueltas.
`    }`	Cierra el ciclo while.
`  }`	Cierra el ciclo for.
`}`	Cierra el método.

Complejidad: O(n log n). El ciclo externo hace n vueltas y en cada una el interno hace ≈ log n: total n × log n.

3. Explicación línea por línea del programa ComparacionAlgoritmos.java (Sección 8)

Este programa mide cuánto tarda (en nanosegundos) cada algoritmo con diferentes tamaños de n, y muestra los resultados en una tabla en consola.

Línea de código	Qué hace
`public class ComparacionAlgoritmos {`	Declara la clase principal. El nombre debe coincidir con el del archivo (ComparacionAlgoritmos.java).
`  public static void main(String[] args) {`	Método main: punto de entrada, la JVM empieza a ejecutar aquí. args recibe argumentos de la línea de comandos (aquí no se usan).
`    int[] tamanos = {100, 1_000, 5_000, 10_000};`	Crea un arreglo con los tamaños de prueba. Los guiones bajos (1_000) son separadores de miles para facilitar la lectura; Java los ignora.
`    System.out.println("n\tO(n) ns\tO(n²) ns");`	Imprime el encabezado de la tabla. \t es un tabulador que separa las columnas.
`    for (int n : tamanos) {`	Ciclo for-each: en cada vuelta n toma uno de los valores del arreglo (100, luego 1.000, luego 5.000 y luego 10.000).
`      long inicioLineal = System.nanoTime();`	Guarda el instante actual en nanosegundos, antes de ejecutar el algoritmo lineal. Se usa long porque el número es muy grande.
`      algoritmoLineal(n);`	Ejecuta el algoritmo O(n) con el tamaño n actual (el valor devuelto se descarta; solo interesa el tiempo).
`      long finLineal = System.nanoTime();`	Guarda el instante en que terminó el algoritmo lineal.
`      long tiempoLineal = finLineal - inicioLineal;`	Calcula el tiempo transcurrido: fin menos inicio, en nanosegundos.
`      long inicioCuadratico = System.nanoTime();`	Guarda el instante de inicio del algoritmo cuadrático.

Línea de código	Qué hace
`algoritmoCuadratico(n);`	Ejecuta el algoritmo O(n²) con el mismo n, para comparar en igualdad de condiciones.
`long finCuadratico = System.nanoTime();`	Guarda el instante en que terminó el algoritmo cuadrático.
`long tiempoCuadratico = finCuadratico - inicioCuadratico;`	Calcula el tiempo del algoritmo cuadrático en nanosegundos.
`System.out.println(n + "\t" + tiempoLineal + "\t" + tiempoCuadratico);`	Imprime una fila: n, tiempo lineal y tiempo cuadrático, separados por tabuladores. El + concatena texto y números.
`}`	Cierra el ciclo for-each.
`}`	Cierra el método main.
`public static long algoritmoLineal(int n) {`	Declara el método O(n): recibe n y devuelve un long.
`long contador = 0;`	Variable que cuenta cuántas iteraciones se hacen; empieza en 0.
`for (int i = 0; i < n; i++) {`	Ciclo simple de n vueltas.
`contador++;`	Suma 1 al contador en cada vuelta (una operación básica). Al final contador vale n.
`}`	Cierra el ciclo.
`return contador;`	Devuelve el contador (n). Devolverlo evita que Java elimine el ciclo por considerarlo inútil.
`}`	Cierra el método.
`public static long algoritmoCuadratico(int n) {`	Declara el método O(n²).
`long contador = 0;`	Contador inicializado en 0 (long porque llegará a n², por ejemplo 100.000.000, que cabe en long).
`for (int i = 0; i < n; i++) {`	Ciclo externo de n vueltas.
`for (int j = 0; j < n; j++) {`	Ciclo interno: n vueltas por cada vuelta del externo.
`contador++;`	Suma 1 al contador; se ejecuta n × n veces. Al final contador vale n².
`}`	Cierra el ciclo interno.
`}`	Cierra el ciclo externo.
`return contador;`	Devuelve el contador (n²).
`}`	Cierra el método.
`}`	Cierra la clase.

Compilación y ejecución (Sección 9)

```
javac ComparacionAlgoritmos.java
java ComparacionAlgoritmos
```

javac compila el código fuente (.java) y genera el bytecode (.class); java ejecuta la clase compilada en la JVM.

4. Registro de resultados (Sección 10)

Tiempos obtenidos al ejecutar ComparacionAlgoritmos (en nanosegundos). Los valores exactos cambian en cada ejecución y computador; lo importante es la tendencia.

Tamaño n	Tiempo O(n) (ns)	Tiempo O(n²) (ns)	¿Cuál fue más rápido?
100	5.175	83.124	O(n)
1.000	11.136	8.926.418	O(n)
5.000	40.579	87.183.972	O(n)
10.000	86.491	252.493.184	O(n)

Observación: con n = 10.000, el algoritmo cuadrático tardó aproximadamente 2.900 veces más que el lineal, y la diferencia crece al aumentar n.

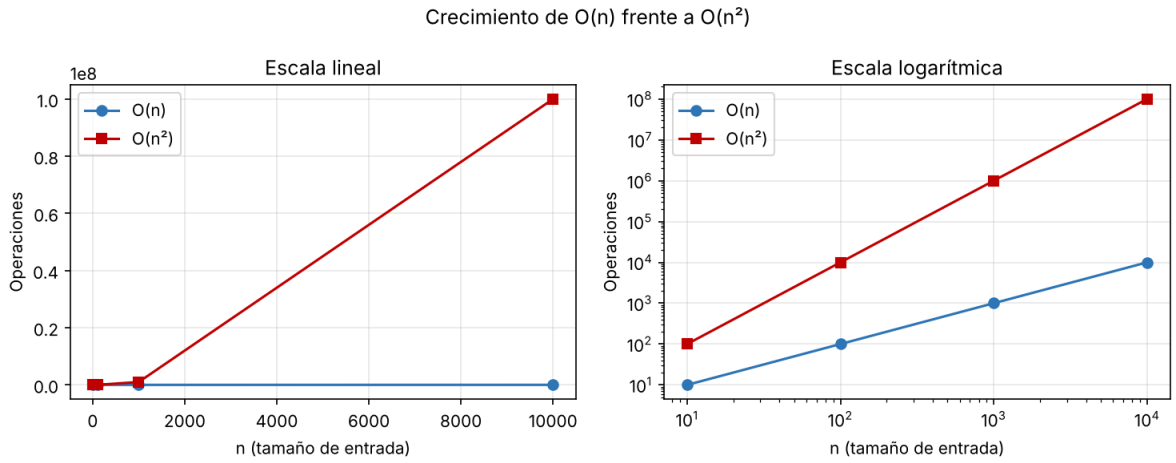
5. Conteo teórico (Parte D, Sección 11)

Logaritmos en base 2, valores aproximados.

n	O(1)	O(log n) aprox.	O(n)	O(n log n) aprox.	O(n²)
10	1	3	10	33	100
100	1	7	100	664	10.000
1.000	1	10	1.000	9.966	1.000.000
10.000	1	13	10.000	132.877	100.000.000

6. Representación gráfica (Parte E, Sección 12)

n	O(n)	O(n²)
10	10	100
100	100	10.000
1.000	1.000	1.000.000
10.000	10.000	100.000.000



Se observa que O(n) crece como una línea recta, mientras que O(n²) se dispara: con n = 10.000 ya requiere 100.000.000 de operaciones frente a 10.000.

7. Preguntas de análisis (Sección 13)

1. ¿Qué diferencia existe entre tiempo de ejecución y complejidad Big O?
El tiempo de ejecución es una medida concreta (segundos o nanosegundos) que depende del computador, el sistema operativo y las optimizaciones de Java. Big O describe cómo crece el número de operaciones al aumentar n, sin depender de la máquina.

2. ¿Por qué O(1) se considera una complejidad eficiente?
Porque el trabajo no cambia aunque aumente el tamaño de los datos: se hace siempre una cantidad fija de operaciones (por ejemplo, acceder a vector[5]).

3. ¿Qué ocurre con un algoritmo O(n) cuando duplicamos el tamaño de entrada?
El tiempo se duplica: 2n / n = 2 veces.

4. ¿Qué ocurre aproximadamente con un algoritmo O(n²) cuando duplicamos n?
El tiempo se cuadruplica: $(2n)^2 / n^2 = 4n^2 / n^2 = 4$ veces. Ejemplo: de 1.000 a 2.000 pasa de 1.000.000 a 4.000.000 de operaciones.

5. ¿Por qué los ciclos anidados suelen relacionarse con O(n²)?
Porque por cada iteración del ciclo externo (n) el interno se ejecuta completo (n), y las operaciones se multiplican: $n \times n = n^2$.

6. ¿Qué característica permite reconocer un algoritmo O(log n)?
Que en cada paso el problema se reduce por un factor constante, normalmente a la mitad (n = n / 2), de modo que el número de pasos es muy pequeño frente a n.

7. ¿Cuál de los dos algoritmos experimentales presentó mayor crecimiento?
El cuadrático O(n²): pasó de 83.124 ns con n = 100 a 252.493.184 ns con n = 10.000, mientras que el lineal pasó de 5.175 ns a 86.491 ns.

8. ¿Los resultados medidos fueron exactamente iguales entre las diferentes ejecuciones? ¿Por qué?
No. Los tiempos varían porque dependen del procesador, la memoria disponible, el sistema operativo, otros procesos en ejecución y las optimizaciones de la JVM (como la compilación JIT). Aun así, la tendencia (O(n²) mucho más lento) se mantiene.

9. Si tuviera que procesar diez millones de registros, ¿qué tipo de complejidad preferiría?
La más baja posible: O(1) u O(log n), o en su defecto O(n) u O(n log n). Un O(n²) implicaría $10.000.000^2 = 100.000.000.000.000$ (10^{14}) de operaciones, inviable en la práctica.

10. ¿Por qué un ingeniero de software debe analizar la eficiencia antes de seleccionar un algoritmo?
Porque un programa puede funcionar bien con pocos datos y degradarse con volúmenes grandes. Analizar la eficiencia evita problemas de rendimiento, ahorra recursos y costos, y permite que el sistema escale.

8. Reto adicional (Sección 14)

```
for (int i = 0; i < n; i++) {
    System.out.println(i);
}

for (int j = 0; j < n; j++) {
    System.out.println(j);
}
```

¿Es $O(n^2)$? No. Es $O(n)$.

Los dos ciclos son consecutivos, no anidados, por lo que sus operaciones se suman, no se multiplican: $n + n = 2n$. Por la regla del término dominante se descartan las constantes, entonces $2n$ se simplifica como $O(n)$. Solo sería $O(n^2)$ si un ciclo estuviera dentro del otro ($n \times n$).

9. Conclusión personal

Con esta actividad comprendí que un programa no es bueno solo porque da el resultado correcto: también importa cómo se comporta cuando crece la cantidad de datos. En el experimento, el algoritmo $O(n^2)$ fue entre 16 y 2.900 veces más lento que el $O(n)$, y la brecha aumentó con cada valor de n. Esto confirma que Big O describe la tendencia de crecimiento y no el tiempo exacto, el cual cambia entre ejecuciones y equipos. También aprendí a identificar complejidades observando la estructura del código: un acceso directo es $O(1)$, un ciclo simple es $O(n)$, dos ciclos anidados son $O(n^2)$, y una reducción a la mitad es $O(\log n)$. Como futuro ingeniero de software, analizar la eficiencia antes de elegir un algoritmo es clave para construir sistemas que escalen con millones de registros.